Worksheet — 1.2 Software & Software Development

OCR A Level Computer Science (H446) — Component 01, Section 1.2 (1.2.1 Operating systems · 1.2.2 Applications generation · 1.2.3 Software development · 1.2.4 Types of programming language)

Name: ___ *Date:* ___

Time: ~30 min. Check against the answer key at the end.

1. Key-term match

Write the letter of the correct definition next to each term.

Term	Answer		Definition
1. Virtual memory	____	A	Excessive paging between RAM and disk that slows the system almost to a halt.
2. Interrupt	____	B	Software emulating a complete computer, or running platform-independent intermediate code.
3. Disk thrashing	____	C	Using secondary storage as if it were RAM to run more/larger programs than RAM allows.
4. ISR	____	D	Dividing memory into fixed-size blocks (pages mapped to frames).
5. Paging	____	E	A signal (hardware or software) that a device/process needs the processor's attention.
6. Virtual machine	____	F	The OS code (handler) run to deal with one particular interrupt.

2. The four stages of compilation (in order)

Fill in the blanks. Write the stages **in the correct order**.

Stage 1 — _____ **analysis:** source code is broken into __; whitespace and comments are removed; a __ table is built.

Stage 2 — __ ***analysis (also called*** ___**):** tokens are checked against the language's __; a __ tree is built; _____ errors are reported here.

Stage 3 — Code _____**:** the parse tree and symbol table are used to produce __ / machine code.

Stage 4 — Code _____**:** the code is refined to run __ or use less __, while producing the same result.

3. Translators — complete the comparison table

	Compiler	Interpreter
When does translation happen?	_____	_____
Output produced?	_____	_____
Program execution speed	_____	_____
Source code needed to run?	_____	_____
How are errors reported?	_____	_____
Best used for…	_____	_____

Q. In one sentence, what does an **assembler** translate, and at roughly what ratio?

4. Match the methodology to the scenario

Methodologies: **Waterfall · Agile · Extreme Programming (XP) · Spiral · RAD**

Write the best-fit methodology next to each scenario.

a) A bank is building a large, expensive air-traffic adjacent system where the cost of failure is huge; management wants formal **risk analysis** at every iteration. → _____

b) A small project for a client whose requirements are **fixed, clear and unlikely to change**, and who wants full documentation at each stage. → _____

c) A start-up needs a working version **fast** and is happy to refine **prototypes** with users; requirements are vague. → _____

d) Safety-critical control software where **code quality is paramount**, so the team uses **pair programming** and test-driven development with a customer on-site. → _____

e) An app whose requirements are **expected to change a lot**; the client wants to see and feed back on **working increments** regularly. → _____

5. Addressing modes

A program executes the instruction `LDA 20`. Memory contents are:

Address	Contents
20	35
35	99
99	12

State the value that is loaded into the accumulator under each addressing mode.

a) **Immediate** addressing: value loaded = _____

b) **Direct** addressing: value loaded = _____

c) **Indirect** addressing: value loaded = _____

d) In one line, explain what **indexed** addressing uses to find the operand, and what it is typically used for.

6. Scheduling & interrupts — short answer

a) Name the scheduling algorithm that gives each process a fixed **time slice (quantum)** in turn, returning unfinished jobs to the back of the queue. State whether it is pre-emptive.

b) **Shortest Job First** minimises average waiting time. Give **one** disadvantage.

c) Define **pre-emptive** scheduling.

d) Put the interrupt-handling steps in order (write 1–6):

Step	Order (1–6)
Load the address of the correct ISR	____
Pop the registers off the stack and resume the original task	____
CPU checks the interrupt register at the end of the F-D-E cycle	____
Run the ISR	____
Push the registers (e.g. PC, ACC) onto the stack	____
Finish the current instruction (if the interrupt is higher priority)	____

e) Why must the registers be saved on a **stack** (LIFO) rather than in a single store?

7. OOP — terms labelling/matching

Match each OOP term to its meaning.

Terms: **Class · Object · Method · Attribute · Inheritance · Encapsulation · Polymorphism**

Meaning	Term
A specific instance of a class.	_____
A template/blueprint defining attributes and methods.	_____
A variable belonging to a class/object (its data/state).	_____
A subroutine belonging to a class (its behaviour).	_____
Bundling data and methods together and hiding the data (private attributes accessed via methods).	_____
A subclass acquiring the attributes/methods of a superclass, and able to add or override them.	_____
The same method name behaving differently depending on the object.	_____

8. Challenge box

> **Challenge.** A team is distributing finished commercial software to customers who must **not** see the source code, and the program must **run quickly** on the user's machine without needing anything else installed.
>
> (i) Should they use a **compiler** or an **interpreter**? Justify with **two** reasons drawn from the table in Q3.
>
> (ii) The OS uses **virtual memory** to let several large programs run at once. Explain why virtual memory does **not** actually increase the amount of RAM, and name the problem caused by **excessive** swapping.

Answer key

1. Key-term match: 1 → C; 2 → E; 3 → A; 4 → F; 5 → D; 6 → B.

2. Stages of compilation: - Stage 1 — **Lexical** analysis: code → **tokens**; remove whitespace/comments; build **symbol** table. - Stage 2 — **Syntax** analysis (also called **parsing**): tokens checked vs **grammar**; **parse** tree built; **syntax** errors reported here. - Stage 3 — Code **generation**: produces **object** / machine code from the parse tree + symbol table. - Stage 4 — Code **optimisation**: refines code to run **faster** / use less **memory**, same result.

3. Translators:

	Compiler	Interpreter
When	All at once, before the program runs	Line by line, at run time
Output	Standalone executable (.exe)	None
Program speed	Fast (already machine code)	Slower (translated each run)
Source needed to run?	No (executable only)	Yes, plus the interpreter
Errors	All reported at the end	Stops at the first error found
Best for	Distributing finished software	Developing/debugging, scripting

Assembler: translates **assembly language** into machine code, at roughly **one mnemonic : one machine instruction** (one-to-one).

4. Methodology match: a) **Spiral**; b) **Waterfall**; c) **RAD**; d) **Extreme Programming (XP)**; e) **Agile**.

5. Addressing modes (worked): For `LDA 20` with [20]=35, [35]=99, [99]=12: - a) **Immediate** — operand *is* the value → loads **20**. - b) **Direct** — operand is the address holding the value → go to address 20, find 35 → loads **35**. - c) **Indirect** — operand is the address of a location holding the *address* of the value → address 20 holds 35; address 35 holds 99 → loads **99**. - d) **Indexed** — operand = base address + contents of the **index register**; used to step through **arrays**.

6. Scheduling & interrupts: - a) **Round robin** — it **is** pre-emptive (the time slice forces a switch). - b) Any one: requires each job's total runtime to be known/estimated in advance; **long jobs can starve** if short jobs keep arriving; not pre-emptive once a job starts. - c) **Pre-emptive** = a running process can be stopped (interrupted) and the CPU reallocated to another process before the first finishes. - d) Order: CPU checks interrupt register (1) → finish current instruction if higher priority (2) → push registers onto stack (3) → load address of correct ISR (4) → run ISR (5) → pop registers off stack and resume (6).

Step	Order
Load the address of the correct ISR	4
Pop the registers off the stack and resume the original task	6
CPU checks the interrupt register at the end of the F-D-E cycle	1
Run the ISR	5
Push the registers (e.g. PC, ACC) onto the stack	3
Finish the current instruction (if the interrupt is higher priority)	2

- e) A stack is **LIFO**, so when a higher-priority interrupt occurs while another ISR is running (**nested interrupts**), the registers are restored in the correct reverse order and every task resumes exactly where it left off.

7. OOP match: instance → **Object**; template/blueprint → **Class**; variable/data → **Attribute**; subroutine/behaviour → **Method**; bundling + data hiding → **Encapsulation**; subclass acquiring superclass features → **Inheritance**; same method name, different behaviour → **Polymorphism**.

8. Challenge: - (i) **Compiler.** Reasons (any two): it produces a **standalone executable** so customers do **not** receive the source code; the program runs **fast** because it is already machine code; the user does **not** need the source or an interpreter installed to run it. - (ii) Virtual memory uses **secondary storage (a swap/page file)** *as if* it were RAM — the physical RAM is unchanged, and disk access is much **slower**

than real RAM. Excessive swapping of pages between RAM and disk causes **disk thrashing**, where the machine spends most of its time paging and grinds to a halt.